



Git Version Control Explained!

Version control systems help programmers keep track of the code they write, and also help them code in collaboration with others. They are very helpful in organising code based on different purposes, versions, in merging different pieces of code, and have many other interesting uses. Git is a new-generation distributed version control system written by Linux Torvalds. This article is a starter guide to Git, and walks through its usage patterns, essential features, internals and how to collaborate with multiple developers using its collaboration tools such as GitHub.

Most people are familiar with centralised version control systems such as Subversion, in which every action requires connectivity to the (Subversion) server. If the central server fails, all collaborators that depend on the server will be affected, and any data access using the central repository will stop. Distributed version control systems (DVCS) do not rely on a central repository server; rather, each user has a full copy of the repository. Whenever the main server fails, you can recover the server from the individual repository copies.

Git combines the best of the earlier DVCS used to maintain the Linux kernel. It is designed on the principles of speed, simplicity, scalability, integrity of data, non-linear development and being fully distributed. Before learning Git, please forget everything you know about version control systems—because Git breaks all the traditional practices and workflows about version control systems that you may be familiar with. Git is modelled like a UNIX filesystem rather than a version control system. Integrity of data is implied by its design. If some files are modified or corrupted, Git itself can identify it by default. Since understanding Git's workflow and internals helps a lot, this article walks you through basic Git terminology and the initialisation steps, then moves to a hands-on workflow, which illustrates how to use it while coding.

Installation

You can install Git from source code, or from your distro's package management system. To install from the latest source, use the following commands:

```
$ wget http://git-core.googlecode.com/files/git-1.7.9.tar.gz
$ tar -xvzf git-1.7.9.tar.gz
$ cd git-1.7.9
$ make install
```

On Ubuntu, a simple `sudo apt-get install git-core` will do the needful.

Basic terminology and concepts

Blobs, trees and commit

These are the three main data structures used in the Git version control system. A *blob* is used to hold a single file. A *tree* is an object used to model a directory that contains files, as well as other sub-directories (a tree holds blobs and sub-trees). A *commit* holds the current state of the repository. It points to a tree.

Let's get more details on the concepts. A blob is a binary file with the file named as the SHA1 hash of that file. A tree is a binary file that stores references to blobs and trees, also named as the SHA1 hash of the tree object. A commit is again named by the SHA1 hash. These hashes are used for naming, in order to ensure the integrity of data. In Git, files are not addressed by name. Everything is content-addressed. Every commit object has a pointer to the parent commit object. From a given commit, you can reverse-traverse by looking at the parent pointers (like a linked list) to check out the history of the commits. If a commit has multiple parent commits, that means the particular commit is created by merging two branches.